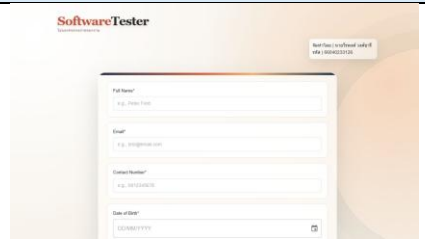


WorkShop 2 “การไหลของข้อมูล (ตัวแปร) ในโปรแกรม :DATA FLOW DETECTIVE WORKSHOP

รหัสนักศึกษา	ชื่อนักศึกษา	รูปภาพนักศึกษา
66040233126	นายวีรพงศ์ วงศ์ชาวี	 A screenshot of a login form titled 'Software Tester'. The form has fields for 'Full Name' (with an example 'e.g. John Doe'), 'Email' (with an example 'e.g. johndoe@co'), 'Contact Number' (with an example 'e.g. 0812345678'), and 'Date of Birth' (with a dropdown arrow). There is a 'Log In' button at the bottom right. A small message at the top right says 'You have successfully logged in! 08/08/2023'.

รูปแบบงาน: งานเดี่ยว (Individual Work)

กติกาสำคัญ

- 1) ไม่ต้องพยายามจำคำศัพท์ทั้งหมดก่อนลงมือทำ
- 2) ใช้ Source Code ของระบบที่ตนเองพัฒนาหรือได้รับมอบหมาย
- 3) ทุกข้อที่ตอบควรมีหลักฐานจาก Source Code / การทดลอง / ภาพหน้าจอ
- 4) หน้า 20–24 เป็น Case Study ของไฟล์บทเรียน ให้เปิดเมื่ออาจารย์สั่ง เพื่อไม่ให้เห็นคำตอบตัวอย่างล่วงหน้า

ข้อกำหนด - ให้นักศึกษานำรูปภาพเว็บไซต์ที่พัฒนาขึ้นมาใส่ที่ตารางด้านล่าง

--

จาก Source Code -> Data Flow -> Def-Use Path -> Test Case

ภารกิจของวันนี้

ให้นักศึกษานำประสบการณ์จากการทำงานในบทบาท Developer และ Tester มาต่อยอดเป็นการทดสอบที่มอง “การไหลของข้อมูลในโปรแกรม” โดยใช้ตัวแปรจริงจาก Source Code ของระบบที่ตนเองพัฒนาขึ้น

กระบวนการเรียนรู้: TRY -> THINK -> DISCOVER -> NAME -> APPLY

ภาพรวมใบงาน

Mission	นักศึกษาจะทำอะไร	หน้าหลักของไฟล์	ผลงานที่ต้องได้
MISSION 1 FIND THE DATA	เลือกตัวแปรจริงจาก Source Code แล้วหา “จุดที่กำหนดค่า” และ “จุดที่นำค่าไปใช้”	หน้า 6–7, 11–12	Def/Use Table + ทำเครื่องหมายใน Source Code
MISSION 2 TRACE THE DATA	ติดตามการไหลของตัวแปร สร้าง Data Flow อย่างง่าย และหา Def-Use Path	หน้า 3–4, 8, 13–17	Data Map + Mini DFG + Def-Use Pair + Paths
MISSION 3 TEST THE PATH	เปลี่ยน Path ให้เป็น Test Case แล้วนำไปทดสอบระบบจริง	หน้า 18–19 และ 20–24 เมื่ออาจารย์สั่ง	Test Case + Evidence + PASS/FAIL + Bug Report/Re-test (ถ้ามี)
<u>จำง่าย</u> Developer = สร้างระบบ Tester = พิสูจน์พฤติกรรมของระบบ Data Flow Tester = ติดตามว่า “ข้อมูลถูกกำหนดค่าและถูกนำไปใช้อย่างไร”			

MISSION 1 — FIND THE DATA

เป้าหมาย: ค้นหาว่า “ข้อมูลของตัวแปรหนึ่งตัว” เริ่มต้นที่ไหน และถูกนำไปใช้ที่ไหน

STEP 1 เลือกตัวแปรจริง

เลือกตัวแปร 1 ตัวจาก Source Code ของเว็บไซต์/โปรแกรมที่คุณกำลังทดสอบ

ชื่อตัวแปร: v.phone / values.phone ชนิดข้อมูล: String

เหตุผลที่เลือกตัวแปรนี้:

เลือก v.phone เพราะรับค่าจากผู้ใช้โดยตรง และมีเส้นทาง Def → ตรวจสอบความถูกต้อง → แสดงข้อผิดพลาดหรือให้ส่งฟอร์มต่อไปได้ชัดเจน

STEP 2 หา Def และ Use

เปิดไฟล์ Chap6-Structural Test_DFT.pdf ที่หน้า 6–7 เพื่อทำความเข้าใจ Define / Use ก่อน จากนั้นกลับมาที่ Source Code ของตนเอง

ลำดับ/บรรทัด	เกิดอะไรขึ้นกับตัวแปร?	ฉันทัดเป็น	หลักฐาน/เหตุผล
constants/formOptions.js:8	createInitialForm กำหนด phone = "	Def	เป็นค่าเริ่มต้นของข้อมูล phone
App.jsx:91–92	set('phone', x) เรียก setV เพื่อกำหนดค่าใหม่ให้ phone	Def	ข้อมูลเบอร์โทรจากผู้ใช้งานบันทึกใน state
App.jsx:332	TextField อ่าน value=(v.phone) เพื่อแสดงในช่องกรอก	Use	นำค่าไปแสดงผล (C-use)
App.jsx:341	onChange อ่าน e.target.value แล้วส่งเข้า set('phone', ...)	Use / Def	ใช้ค่าที่ผู้ใช้พิมพ์ แล้วกำหนดค่าใหม่ให้ phone
validateForm 21–25	อ่าน values.phone เพื่อตรวจว่า รูปแบบ และจำนวนตัวเลข	Use	ใช้ในเงื่อนไข (P-use) เพื่อเลือกผลลัพธ์

STEP 3 แยก P-use / C-use

เปิดหน้า 11-12 ของไฟล์ แล้วลองจัดประเภท Use ที่พบใน Source Code ของคุณ

ตำแหน่ง	ตัวแปร	P-use หรือ C-use?	เหตุผลสั้น ๆ
App.jsx:332	v.phone	C-use	ใช้เป็นค่า value ของ TextField เพื่อแสดงเบอร์โทร
validateForm 21	values.phone	P-use	ตรวจสอบว่าเบอร์โทรว่างหรือไม่ด้วย if
validateForm 22	values.phone	P-use	ตรวจสอบความยาว อักขระที่อนุญาต และการมีตัวเลข
validateForm 24	values.phone	P-use	นับตัวเลขแล้วเปรียบเทียบกับว่าครบ 10 หลักหรือไม่
ค่าของ v.phone เดินทางจากช่องกรอกไปยัง state แล้วถูกตรวจสอบด้วยเงื่อนไข ก่อนนำผลไปกำหนดข้อความผิดพลาดและการส่งฟอร์ม			

MISSION 2 — TRACE THE DATA

เป้าหมาย: เปลี่ยนจากการมอง Source Code เป็นรายบรรทัด -> มอง “เส้นทางของข้อมูล”

STEP 1 วาด Data Map อย่างง่าย

ยังไม่ต้องวาด DFG แบบเป็นทางการ ให้เริ่มจากลูกศรแสดงการเดินทางของตัวแปร

ตัวอย่างรูปแบบการเขียน (ไม่ใช่คำตอบของระบบคุณ)

Def -> Use -> Use -> Use

เช่น จุดที่กำหนดค่า -> จุดที่ตรวจสอบ -> จุดที่คำนวณ/แสดงผล

วาด Data Map ของตัวแปรที่เลือกลงในพื้นที่ด้านล่าง

Data Map: TextField onChange → set('phone', e.target.value) → setV({ ...v, phone: x }) → v.phone → validateForm(v) → errors.phone → FieldSection / Snackbar

STEP 2 เรียนรู้ Data Flow Testing

เปิดหน้า 3-4 ของไฟล์ Chap6 เพื่อค้นหาว่า Data Flow Testing สนใจอะไร จากนั้นตอบด้วยภาษาของตนเอง

คำถาม	คำตอบของฉัน
Data Flow Testing มองอะไรในโปรแกรม?	ติดตามว่าค่าของตัวแปรถูกกำหนดที่ใด ถูกใช้ที่ใด และเดินทางผ่านเส้นทางใดในโปรแกรม
ทำไมการติดตามตัวแปรจึงช่วยในการทดสอบ?	ช่วยสร้าง Test Case จากจุดที่ข้อมูลเปลี่ยนค่าและจุดที่ข้อมูลถูกตรวจสอบ จึงพบกรณีขอบเขตหรือเงื่อนไขที่พลาดได้

STEP 3 เรียนรู้ Def-Use Pair และ Def-Clear Path

เปิดหน้า 8 ของไฟล์ แล้วนำแนวคิดมาจับคู่ Def กับ Use ของตัวแปรที่คุณเลือก

Def จุดที่	Use จุดที่	เป็น Def-Use Pair หรือไม่?	เหตุผล
D1: createInitialForm phone = ""	U1: TextField value={v.phone}	ใช่	ค่าเริ่มต้นถูกอ่านเพื่อแสดงช่องกรอก

Def จุดที่	Use จุดที่	เป็น Def-Use Pair หรือไม่?	เหตุผล
D1: createInitialForm phone = ""	U2: validateForm ตรวจค่าว่าง	ใช่	ไม่มีการกำหนด phone ใหม่ก่อนตรวจ
D2: set('phone', x)	U3: TextField value={v.phone}	ใช่	ค่าที่กรอกถูกแสดงกลับในช่องเดิม
D2: set('phone', x)	U4: validateForm ตรวจรูปแบบ	ใช่	ค่าจาก input ถูกใช้ในเงื่อนไข validation

MISSION 2 — TRACE THE DATA (ต่อ)

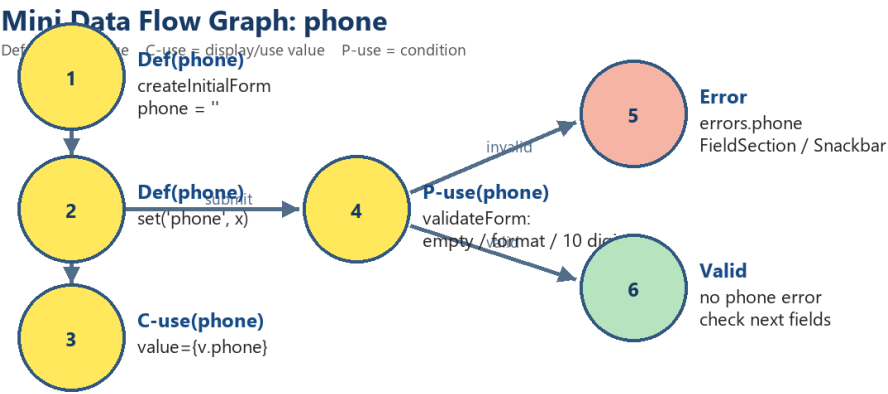
STEP 4 ค้นหา “3 ขั้นตอน” ของ Data Flow Testing

เปิดหน้า 13 ของไฟล์ แล้วสรุป 3 ขั้นตอนด้วยคำของคุณเอง

ขั้นตอน	สิ่งที่ฉันเข้าใจ
1	สร้าง Data Flow Graph จากโค้ด และระบุโน้ต Def, C-use และ P-use
2	หา Def-Use Paths ที่เชื่อมจากการกำหนดค่าไปยังจุดที่ใช้ค่า โดยไม่มีการกำหนดค่าซ้ำระหว่างทาง
3	ออกแบบ Test Case จากแต่ละ Path แล้วเปรียบเทียบ Expected Result กับ Actual Result

STEP 5 สร้าง Mini Data Flow Graph (DFG)

เปิดหน้า 14–15 เพื่อดูแนวคิด DFG แล้วสร้าง DFG ของตัวแปรของคุณเอง



STEP 6 สร้าง Def-Use Paths

เปิดหน้า 16–17 แล้วหาเส้นทางที่เชื่อมจาก Def ไปยัง Use ตามกระแสดำเนินการของโปรแกรม

Path	เส้นทางที่ฉันทพบ	ชนิด Use
Path-1	เริ่มต้น phone = " → validateForm ตรวจสอบค่าว่าง	P-use: !values.phone.trim()
Path-2	กรอก 9 หลัก → set phone → นับจำนวนตัวเลข	P-use: จำนวนเลข ≠ 10
Path-3	กรอก 10 หลัก → set phone → validation ผ่าน	P-use: ทุกเงื่อนไขเป็น false
Path-4	กรอกอักขระปน → set phone → ตรวจสอบรูปแบบ	P-use: regex ไม่ผ่าน

คำเตือน

ไฟล์บทเรียนระบุแนวทางให้ใช้ Path ที่หาได้ไปสร้าง Test Cases ในขั้นต่อไป อย่าเหมารวมว่า “จำนวน Path = จำนวน Test Case” เป็นกฎสากลของการทดสอบทุกประเภท ให้ยึดกระบวนการและตัวอย่างตามบทเรียนนี้เป็นหลัก

MISSION 3 — TEST THE PATH

เป้าหมาย: นำ Def-Use Path ที่สร้างได้ -> เปลี่ยนเป็น Test Case -> ทดสอบระบบจริง

STEP 1 Path -> Test Data

เปิดหน้า 18-19 ของไฟล์ เพื่อดูการนำ Path ไปสร้าง Test Case

เลือกอย่างน้อย 2 Paths ของคุณ แล้วออกแบบข้อมูลทดสอบ

Test Case ID	Path	Test Data / Input	Expected Result
TC-01	Path-1	phone = "	ไม่ส่งฟอร์มและแสดง 'กรุณากดกรอกเบอร์โทร'
TC-02	Path-2	phone = '081234567'	ไม่ส่งฟอร์มและแสดง 'กรุณากดกรอกเบอร์โทรให้ครบ 10 หลัก'
TC-03	Path-3	phone = '0812345678'	ไม่มี error ของ phone และระบบตรวจสอบบังคับต่อไป
TC-04	Path-4	phone = '08123abc78'	ไม่ส่งฟอร์มและแจ้งว่าใช้ได้เฉพาะตัวเลข + - และช่องว่าง

STEP 2 ลงมือทดสอบระบบจริง

นำ Test Data ไปกรอก/ส่งเข้าเว็บไซต์ของคุณ แล้วบันทึกผลที่เกิดขึ้นจริง

TC ID	Actual Result	PASS / FAIL	Evidence
TC-01	แสดงข้อความตรงตามที่ได้คาด และไฟก๊สของ Phone	PASS	ภาพทดสอบ / npm test
TC-02	แสดงข้อความให้กรอกครบ 10 หลัก และไฟก๊สของ Phone	PASS	ภาพทดสอบ / npm test
TC-03	ไม่มีข้อความผิดพลาดของ Phone แล้วระบบไปตรวจสอบของ Date of Birth	PASS	npm test
TC-04	แสดงข้อความแจ้งรูปแบบเบอร์โทรไม่ถูกต้อง	PASS	npm test

STEP 3 ถ้าพบ FAIL ให้ตรวจสอบว่าเป็น Bug หรือไม่

อย่าเขียนเพียงว่า “Fail = Bug” ให้เปรียบเทียบ Expected Result กับ Actual Result และตรวจสอบหลักฐาน

คำถาม	คำตอบ
Expected กับ Actual ต่างกันตรงไหน?	Expected และ Actual ตรงกันทั้ง 4 Test Cases: ข้อมูลผิดถูกปฏิเสธ ส่วนข้อมูล 10 หลักผ่าน validation ของ Phone
มีหลักฐานอะไรยืนยัน?	ผลการทดสอบอัตโนมัติ 79 กรณี และการทดสอบหน้าเว็บจริงที่แสดงข้อความ/ไฟก๊สในของ Phone
คิดว่าเป็น Defect/Bug หรือไม่? เพราะอะไร?	ไม่เป็น Defect ในเวอร์ชันปัจจุบัน เพราะผลลัพธ์ตรงตามข้อกำหนดเบอร์โทร 10 หลัก

เมื่อพบ Bug — BUG REPORT -> RE-TEST

ถ้า Test Case ใด FAIL และมีเหตุผลเพียงพอ ให้ใช้ข้อมูลจากการทดลองเขียน Bug Report

หัวข้อ	รายละเอียด
Bug ID	N/A - ไม่พบ FAIL ใน Paths ที่เลือก
Bug Title	ไม่มี Bug จากการทดสอบ v.phone ในเวอร์ชันปัจจุบัน
Test Case	TC-01 ถึง TC-04
Steps to Reproduce	ทดสอบข้อมูลว่าง, 9 หลัก, 10 หลัก และตัวอักษรบน ตาม Test Case
Expected Result	ข้อมูลผิดต้องถูกปฏิเสธ ข้อมูล 10 หลักต้องผ่าน validation
Actual Result	ผลลัพธ์ตรงตาม Expected Result ทุกกรณี
Severity / Impact	N/A
Evidence	ผล npm test 79 กรณี และภาพหน้าเว็บไซต์

RE-TEST หลังแก้ไข

รายการ	ผลการ Re-test
ใช้ Test Data เดิมหรือไม่?	ไม่ใช่ เพราะไม่พบ Bug ที่ต้องแก้ไขใน Path ที่เลือก
Actual Result หลังแก้ไข	N/A
ผลสุดท้าย	N/A - ไม่ต้อง Re-test
หลักฐาน	ผล TC-01 ถึง TC-04 เป็น PASS

เชื่อมโยงกับงานก่อนหน้า

ก่อนหน้านี้คุณทดสอบจาก “Input -> Expected -> Actual”

วันนี้คุณเพิ่มมุมมองใหม่คือ “ตัวแปร -> Def -> Use -> Path -> Test Case”

ดังนั้น Test Case ไม่ได้เกิดจากการเดาสุ่ม แต่สามารถสร้างจากโครงสร้างของโปรแกรมได้

CASE STUDY — ใช้เมื่ออาจารย์สั่ง

คำสั่งสำคัญ		
ไฟล์ Chap6-Structural Test_DFT.pdf หน้า 20–24 มีตัวอย่าง Case Study ให้เปิดตามคำสั่งของอาจารย์หลังจากทำ Mission 1–3 แล้ว เป้าหมายคือ “เปรียบเทียบกระบวนการคิด” ไม่ใช่คัดลอกคำตอบ		
หน้าของไฟล์	หน้าที่ของหน้า	สิ่งที่นักศึกษาควรสังเกต
3–4	แนวคิด Data Flow Testing	DFT สนใจการไหล/การใช้งานของข้อมูลในโปรแกรม
6–7	Define / Use	จุดใดเป็นการกำหนดค่า และจุดใดเป็นการใช้ค่า
8	Def-Use Pair / Def-Clear Path	การเชื่อมจาก Def ไป Use โดยไม่ถูกกำหนดค่าใหม่ระหว่างทาง
11–12	P-use / C-use และตัวอย่าง Path	การใช้ตัวแปรในเงื่อนไขและการคำนวณ/ประมวลผล
13	กระบวนการ 3 ขั้น	DFG -> Paths -> Test Cases
14–15	Data Flow Graph	เปลี่ยนโค้ดเป็นโครงสร้างการไหลของข้อมูล
16–17	การสร้าง Path	สร้าง Path ตามความสัมพันธ์ Def-Use
18–19	การสร้าง Test Cases	นำ Path ไปสร้างข้อมูลทดสอบและ Expected Result
20–24	Case Study	ดูการเชื่อมโยง Def/Use -> Path -> Test Cases ในตัวอย่าง

เปรียบเทียบกับงานของฉัน

ประเด็น	Case Study ในไฟล์	งานของฉัน
ตัวแปรที่ติดตาม	Usage	v.phone / values.phone
Def	input(Usage)	createInitialForm และ set('phone', x)
Use	เงื่อนไข if และคำนวณ Bill	TextField และเงื่อนไข validateForm
Def-Use Pair	Def(Usage) → P-use(Usage)	Def(v.phone) → P-use(values.phone)
Path	ตามค่าของ Usage ในแต่ละช่วง	ว่าง / 9 หลัก / 10 หลัก / รูปแบบผิด
Test Case	ค่าขอบเขต Usage	TC-01 ถึง TC-04

REFLECTION — ฉันเข้าใจ Data Flow Testing แล้วหรือยัง?

1. ก่อนทำกิจกรรมนี้ ฉันคิดว่า Test Case เกิดขึ้นจากอะไร?

ก่อนทำกิจกรรมนี้ ฉันคิดว่า Test Case เกิดจากการลองกรอกข้อมูลหลายแบบ แต่ยังไม่ได้เชื่อมโยงกับตัวแปรและเส้นทางใน Source Code อย่างชัดเจน

2. หลังทำกิจกรรมนี้ ฉันเห็นความสัมพันธ์ระหว่าง Source Code กับ Test Case อย่างไร?

หลังทำกิจกรรมนี้ ฉันเห็นว่า Source Code บอกได้ว่าข้อมูลเริ่มต้นและเปลี่ยนค่าอย่างไร ส่วน Test Case สามารถออกแบบจากเส้นทาง Def → Use ของข้อมูลนั้นได้

3. Def และ Use ช่วยให้เราค้นหาเส้นทางทดสอบได้อย่างไร?

Def ช่วยหาแหล่งที่ข้อมูลถูกกำหนดค่า ส่วน Use ช่วยหาจุดที่ข้อมูลถูกนำไปแสดงผลหรือใช้ในเงื่อนไข จึงทำให้เลือก Path ที่ควรทดสอบได้เป็นระบบ

4. จุดไหนของกิจกรรมที่ฉันยังไม่เข้าใจ และต้องการให้อาจารย์อธิบายเพิ่ม?

ยังต้องการให้อาจารย์อธิบายเพิ่มเรื่องการวาด DFG ของ React state ที่มีการอัปเดตแบบ asynchronous และวิธีเลือก Def-Use Pair ที่สำคัญเมื่อมีหลายตัวแปร

5. ถ้าต้องกลับไปทดสอบเว็บไซต์ของตนเองอีกครั้ง ฉันจะปรับวิธีคิด/วิธีทดสอบอย่างไร?

ฉันจะเริ่มจากเลือกตัวแปรสำคัญ เช่น phone, dob หรือ file แล้วติดตาม Def, Use และเงื่อนไข validation ก่อนสร้าง Test Case ทั้งค่าปกติ ค่าขอบเขต และค่าที่ผิดปกติ

สิ่งที่ควรจำ

“อย่าเริ่มจากการสุ่ม Test Case อย่างเดียว

ให้เริ่มจากการถามว่า ข้อมูลถูกกำหนดที่ไหน -> ถูกใช้ที่ไหน -> เดินทางผ่าน Path ไหน -> แล้วจึงสร้าง Test Case”

CHECKLIST ก่อนส่งงาน

☐ เลือกตัวแปรจริงจาก Source Code ☐ ระบุ Def และ Use ☐ แยก P-use / C-use เมื่อพบ ☐ สร้าง Data Map / Mini DFG ☐ หา Def-Use Pair และ Path ☐ สร้าง Test Case จาก Path ☐ บันทึก Expected / Actual / PASS/FAIL ☐ แนบ Evidence ☐ ทำ Bug Report และ Re-test เมื่อพบ Defect